JAVA MINI PROJECT REPORT



An Android-Based Health Record and Reminder And Resolver Management Application

Submitted by

Agnel Faria — Roll No. 2605

Amlisha Cardoso — Roll No. 2608

Vaishnavi Sharma — Roll No. 2666

Declaration

We, the undersigned students, present this report for the Java mini project titled “MediSync: An Android-Based Health Record and Reminder Management Application.” The report describes the problem addressed, requirements, design, implementation approach, testing activities, limitations, and future improvements of the application developed by our group.

Name	Roll number	Signature
Agnel Faria	2605	
Vaishnavi Sharma	2666	
Amlisha Cardoso	2608	

Acknowledgement

We would like to acknowledge everyone who supported us during the planning and development of MediSync. Working on a mobile healthcare application gave us an opportunity to apply programming concepts to a practical problem and understand how several small features must work together to create a useful product.

We are grateful to our faculty for providing a learning environment in which we could practise Java programming, Android interface development, database operations, testing, and project documentation. The project encouraged us to think beyond individual screens and consider the full user journey, from account creation to storing health details and receiving reminders.

We also appreciate the contribution of our teammates. A group project requires discussion, coordination, testing, and willingness to revise decisions when implementation reveals an issue. Developing MediSync helped us understand the importance of dividing a large task into manageable modules while keeping those modules consistent with the overall design.

Finally, we acknowledge the learning resources and developer documentation that helped us understand Android components and local data storage. This report records our current understanding of the project and the work still required to make the documentation fully match the final application.

Abstract

MediSync is an Android-based health record and reminder management application designed to help users keep selected health-related information organised in one place. The application was developed using Java for application logic, XML for Android user-interface layouts, Android Studio as the development environment, and SQLite for local data storage. Its main idea is captured by three concepts: Record, Remind, and Resolve.

The Record module supports a basic health profile, medicine details, medical report metadata, and emergency contacts. The Remind module supports medicine and appointment reminders using Android alarm and notification mechanisms. Resolve includes a Problem Resolver that displays predefined general suggestions and MediBot, an offline assistant that provides prepared responses to supported questions. Additional functions include account management, a Remember Me option, and opening the device phone dialer with a saved emergency number.

The project focuses on practical organisation and ease of use. SQLite provides local storage, while user IDs associate records with the relevant account. The supplied project notes describe tests for authentication, record operations, reminders, navigation, Problem Resolver, MediBot, and separation of records between accounts. These test results should be reconfirmed on the final build and supported by screenshots before formal submission.

MediSync is a personal information organiser and general-information tool. It does not diagnose diseases, prescribe treatment, or replace consultation with a qualified health professional. The report also discusses limitations including local-only storage, prepared chatbot responses, limited medical document handling, and the need for stronger security before wider deployment.

Executive Summary

MediSync was developed to address everyday difficulties such as remembering medicine schedules, keeping health details organised, tracking appointment information, and reaching an emergency contact quickly. Rather than building a large hospital-management system, the project focuses on a personal Android application with a clear dashboard and a defined set of practical modules.

The application has three main areas. Record manages profile details, medicine entries, report metadata, and emergency contacts. Remind lets users maintain medicine and appointment information and schedule notifications. Resolve offers predefined suggestions for selected common problems. MediBot complements these features with prepared offline responses, suggested questions, voice input, and text-to-speech output. The dashboard also links to Emergency and Account Management.

The implementation uses Java and XML in Android Studio. SQLite stores user records locally, while SharedPreferences maintains selected session information and preferences. AlarmManager and a BroadcastReceiver support scheduled reminder handling, and Android Intents open the phone dialer with a saved number. The modules were integrated and subjected to functional and navigation checks.

The main outcome is an integrated mobile application that combines basic record management and reminders, while demonstrating Android and database concepts. The current version has limits: no automatic cloud synchronisation, prepared chatbot responses, and no advanced database encryption. Future work could add secure backup, actual report-file uploads, improved accessibility, and authorised clinic integration.

Table of Contents

Page numbers correspond to this draft and must be checked again after the real application screenshots are inserted.

Section	Page
Title page	1
Declaration	2
Acknowledgement	2
Abstract	2
Executive Summary	3
List of Figures and Tables	4
Abbreviations and Terms	4
Chapter 1: Introduction	5
Chapter 2: Project Planning and Brainstorming	8
Chapter 3: Requirement Analysis	11
Chapter 4: System Design	15
Chapter 5: Tools and Technologies	21
Chapter 6: Implementation of Modules	25
Chapter 7: Testing and Validation	28
Chapter 8: Screenshots of the Working Product	33
Chapter 9: Limitations of MediSync	42
Chapter 10: Future Scope	43
Chapter 11: Challenges and Learning Outcomes	44
Conclusion	47
References and Technical Resources	48
Appendix A: Final Test Execution Template	49
Appendix B: Suggested Demonstration Walkthrough	50
Appendix C: Final Report Completion Checklist	51

List of Figures and Tables

Figures illustrate the system's logical structure and workflows. The diagrams are based on the supplied project description and should be checked against the actual Java source and database schema before final submission. Screenshots of the working application have not yet been supplied and are shown as placeholders in Chapter 8.

Figure / Table	Title
Figure 2.1	Development lifecycle flowchart
Figure 4.1	System architecture
Figure 4.2	Application navigation flow
Figure 4.3	Use case diagram
Figure 4.4	Authentication activity flow
Figure 4.5	Record management activity flow
Figure 4.6	Reminder notification sequence
Figure 4.7	Logical entity-relationship diagram
Figure 4.8	Logical component diagram
Figure 4.9	CRUD data flow
Figure 7.1	Testing process
Table 3.1	Functional requirements
Table 3.2	Non-functional requirements
Table 7.1	Functional test register
Table 8.1	Screenshot evidence checklist

Abbreviations and Terms

The following abbreviations appear throughout the report.

Term	Meaning
Android	Mobile application platform
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ERD	Entity-Relationship Diagram
UI	User Interface
ID / PK / FK	Identifier / Primary Key / Foreign Key
MediBot	Offline assistant that returns prepared health-related responses
NFR	Non-Functional Requirement
SQLite	Embedded relational database used for local storage
XML	Markup format used for Android interface layouts
AlarmManager	Android API used to schedule alarm events
BroadcastReceiver	Android component used to react to reminder events
Intent	Android messaging object used to request an action such as opening a dialer

Chapter 1: Introduction

Healthcare information is often spread across prescription packets, paper reports, contact lists, messages, and memory. A person may need to remember the schedule of a medicine, keep track of a doctor appointment, locate report details, or reach a family member quickly. When these details are scattered, they can be difficult to keep current and accessible.

MediSync is an Android application created to bring selected personal health details and reminders into a single interface. It is not a replacement for a hospital information system, an electronic health record platform, or a medical professional. Its purpose is to provide a personal organiser for records and reminders, alongside limited general guidance for supported everyday questions.

The application is based on the concept “Record, Remind, Resolve.” Record helps organise information. Remind supports medicine and appointment notifications. Resolve presents predefined suggestions through Problem Resolver and prepared answers through MediBot. Emergency contact access and account management support the main functions.

This report describes the path from identifying the problem to analysing requirements, designing the application, implementing modules, storing information, testing behaviour, and identifying future improvements. It should remain aligned with the final code and installed application.

1.1 Background and Context

Many mobile applications focus on a single aspect of personal health management, such as reminders, activity tracking, document storage, or appointment booking. A mini project can explore a smaller integrated version by combining everyday functions in one Android application. The challenge is not only to display forms, but also to save information, retrieve the correct records, and provide feedback when the user interacts with the app.

MediSync uses local persistence for its main records. This reduces dependence on network availability and is suitable for a project that demonstrates Android interface development and data operations. SQLite provides structured storage,

while user identifiers associate stored records with individual accounts. Android capabilities support notifications, dialing, and speech-related interaction.

The problem is practical rather than clinical. People may forget a planned dose or appointment, or have difficulty locating information they previously recorded. A single dashboard can reduce the number of places a user needs to check. Selectable categories and prepared responses can also reduce typing.

Because health information is personal, even an educational application should be careful about its limits. Local storage, a login screen, and user-specific queries are useful foundations, but they do not automatically guarantee strong privacy or production-grade security. The current application is best understood as a prototype with a defined feature set.

1.2 Problem Statement

Users may have difficulty keeping medicine information and medical report details together, remembering medicine schedules and appointments, and finding emergency contact details quickly. Applications that require extensive free-text input may also be inconvenient for users who prefer simple selections. General health information can be difficult to access quickly when there is no internet connection.

The project problem is to design and implement an Android application that lets an individual maintain selected health-related details, schedule medicine and appointment reminders, store emergency contacts, and access limited general guidance through a simple interface, while keeping records associated with the correct local account.

The boundaries are important. The application is not intended to diagnose symptoms, generate clinical decisions, prescribe medication, store an entire medical archive, or serve as an emergency service. A reminder is a prompt for the user; it is not a guarantee that medication is appropriate or that a notification will always appear under every device configuration.

Success depends on whether the modules behave as intended, data is saved and retrieved correctly, navigation is understandable, and limitations are communicated clearly.

1.3 Objectives

The objectives for MediSync are to:

- Provide registration, login, logout, and basic account settings.
- Store and display a basic health profile with age, blood group, allergies, and notes.
- Allow users to add, view, edit, and delete medicine records with dosage, frequency, dates, and reminder time.
- Maintain medical report metadata such as title, doctor, hospital, date,and notes.
- Save emergency contact details and open the phone dialer for a selected number.
- Allow users to record medicine and appointment reminders and receive Android notifications when events are triggered.
- Offer predefined suggestions through Problem Resolver and prepared offline answers through MediBot.
- Associate health-related records with the correct user ID and test separation across accounts.
- Apply Java, XML, SQLite, and Android components in a modular group project.

1.4 Project Scope

The scope covers personal information management on an Android device. It includes authentication, profile details, medicine entries, report metadata, emergency contacts, appointment details, reminders, predefined guidance, the offline MediBot interface, and account settings. The app is designed around local database operations rather than a central server.

Within this scope, the application performs CRUD operations for supported record types. It uses the logged-in user identity to filter related data. The Reminder module combines saved details with Android alarm and notification mechanisms. The Emergency module opens the phone dialer with a saved number and requires the user to place the call manually.

The current scope does not include diagnosis, prescription generation, telemedicine, live doctor consultation, insurance processing, pharmacy purchasing, cloud synchronisation, hospital staff workflows, or complete medical document storage. Those capabilities are not described as implemented in the supplied project notes.

Problem Resolver and MediBot provide predefined general information only. Future versions should have any medical content reviewed by qualified

professionals and include clear guidance to seek professional care when appropriate.

1.5 Significance and Intended Users

The intended users are individuals who want a simple local place to record basic health details and set reminders. The interface should expose everyday tasks through labels, buttons, and selectable fields rather than require the user to understand databases or Android APIs. A family member helping another person may also find a central place for contact and reminder information useful, although the current application is not a caregiver-management system.

From a software development perspective, the project integrates event handling, validation, database access, local preference storage, notifications, and navigation through Android Intents. It also demonstrates modular decomposition: one module may manage profile information while another manages reminders, yet both depend on consistent user identity and database behaviour.

MediSync does not guarantee medicine adherence or the safety of a health decision. Reminders may be affected by device settings and permissions. The application is useful as a learning prototype and organisational aid only when its limitations are understood.

A key measure of usefulness is whether a user can complete common tasks with little confusion: save a record, find it later, edit or delete it, set a reminder, and return to the dashboard.

Chapter 1 Summary

MediSync is positioned as a personal health information organiser with reminders and limited offline guidance. Its three concepts—Record, Remind, and Resolve—provide a simple description of its functionality.

The next chapter documents project planning and brainstorming. The supplied material describes the final features but does not contain meeting minutes or a dated record of every idea considered. The brainstorming narrative is therefore a suggested reconstruction based on the delivered scope.

Chapter 2: Project Planning and Brainstorming

Planning focused on everyday health-management tasks that could realistically be addressed in a mini project. The final scope combines record keeping, reminders, general suggestions, and quick access to an emergency contact. Development was then divided into requirements, design, implementation, database integration, notification integration, testing, and final integration.

A good project report records how an idea was selected, not only the final feature list. The available notes do not include dated brainstorming minutes or a definitive list of rejected proposals. The following pages provide a planning narrative based on the implemented scope and identify decisions the team should validate. Do not treat an example as an actual team decision unless it reflects what happened.

Useful criteria for choosing features include user value, feasibility within the project timeline, offline operation, availability of Android APIs, data requirements, ease of demonstration, testing effort, and safety boundaries. These are particularly important for a healthcare-related application, where an ambitious feature may be misleading if it cannot be tested or supported properly.

2.1 Identifying the Need

The final application points to five needs: storing medicine details, keeping report-related information, remembering medicine and appointment schedules, retrieving emergency contacts, and accessing general everyday guidance without lengthy typing. These needs are related because they concern organisation and timely access to information.

Useful planning questions include: What information should be saved locally? Which tasks require a reminder rather than only a record? Which features can be implemented offline? How can users find a function without navigating through many menus? How can the app avoid showing one account's information while another account is active?

The resulting structure groups personal information under Record, scheduled events under Remind, and predefined suggestions and MediBot under Resolve.

Emergency and Account Management are also available from the dashboard. This makes the product's primary purpose visible without requiring the user to understand the internal architecture.

For a final report, this section can be strengthened with genuine evidence such as dated meeting notes, a whiteboard photograph, or a list of initial feature requests. Do not include private messages unrelated to the project.

2.2 Brainstorming Framework and Selection Criteria

Brainstorming is most productive when ideas are first recorded openly and then compared using the same criteria. For MediSync, suitable criteria include user value, feasibility within a mini-project timeline, availability of Android APIs, offline operation, data requirements, ease of demonstration, testing effort, and safety boundaries.

An idea with high user value but excessive complexity may need to be reduced. Rather than building cloud synchronisation and multi-device accounts, the team can begin with local SQLite storage. Rather than generating medical advice, a small student project can provide prepared general responses and clearly communicate their limitations.

The comparison should favour transparent functionality that can be demonstrated and tested. A feature is not complete merely because its screen exists: associated data must be saved or processed correctly, error conditions should be considered, and users should be able to navigate back to the rest of the application.

The matrix below is a reasoned comparison of possible project directions based on the delivered features. It is a draft planning artefact, not evidence that a formal scoring meeting took place.

Candidate idea	Value	Main considerations	Draft outcome
Personal health organiser	Keeps details together	Forms, SQLite tables, CRUD operations	Included as Record
Medicine and appointment reminders	Supports recall	Date/time handling, alarms, notification permissions	Included as Remind
Symptom diagnosis	Could address health questions	High risk and outside the project scope	Reduced to general suggestions

Offline Q&A assistant	Quick supported answers	Prepared replies are feasible; broad understanding is not	Included as MediBot
Cloud storage and sync	Access from multiple devices	Backend, security, network and conflict handling	Deferred
Clinic administration portal	Supports clinics	Large multi-role system and access-control needs	Out of current scope
Emergency contact shortcut	Quick access to a number	Saved contact and dialer Intent	Included

2.3 Why MediSync Was Selected

The selected concept brings together a health record organiser, reminder management, and basic general guidance. The name MediSync suggests coordination of selected health information and everyday tasks; it should not be interpreted as a claim of cloud synchronisation, because the current version uses local storage and does not automatically sync records across devices.

The “Record, Remind, Resolve” structure gives users a concise mental model. Record captures information that may need to be checked later. Remind schedules a future prompt. Resolve provides access to limited predefined suggestions and prepared assistant responses. Emergency and Account Management support these main areas.

The concept is feasible for a mini project because it can be divided into modules with clear boundaries and demonstrable outcomes. The database and user identifier connect the record types. AlarmManager and a BroadcastReceiver address timed reminders. Intents support the dialer action. The interface can remain relatively simple while still demonstrating several Android and Java concepts.

The application name, final modules, and technology stack are confirmed by the supplied project material. The exact group discussion and scoring process still need confirmation if they are to be presented as an actual historical record.

2.4 Accepted, Reduced, and Deferred Ideas

The delivered feature set includes local health records, medicine and appointment reminders, emergency contact access, and general guidance. These

have observable outcomes that can be tested: information can be saved and retrieved, a reminder can be scheduled, a dialer can be opened, and selecting a supported problem can display a prepared suggestion.

Some feature ideas have a limited form. Problem Resolver uses selectable categories and predefined suggestions rather than diagnosis. MediBot supplies prepared offline responses rather than generative AI. Medical report management saves metadata rather than a complete document archive. Emergency access opens the dialer but does not automatically place a call.

Cloud backup, multi-device synchronisation, medical-document uploads, clinic administration, stronger authentication, database encryption, and advanced language understanding are future improvements in the supplied project notes. They must not be listed as implemented features.

This distinction is important in project reporting. A smaller feature that works and is tested is a stronger result than a large promised capability that cannot be supported by the code.

Decision type	Examples	Reason / boundary
Accepted	Local records, reminders, emergency dialer	Clear use cases and testable outcomes
Reduced	Problem Resolver, MediBot, report metadata	Keeps scope feasible and avoids unsupported medical claims
Deferred	Cloud sync, uploads, clinic portal, advanced security	Needs additional architecture, testing, and safeguards

2.5 Development Phases and Work Tracking

The project notes describe a phased development process. Requirement analysis identified the information users should manage. System design defined modules, navigation, database structure, and layouts. Application development created the dashboard and feature screens. Database integration added SQLite operations. Reminder integration added AlarmManager and BroadcastReceiver behaviour. Testing and debugging checked the main workflows. Final integration connected the modules.

A phased structure supports incremental implementation. The dashboard establishes navigation; Record then connects to the data model; reminder screens use the same user identity and database conventions; and Resolve,

MediBot, Emergency, and Account Management are integrated and tested. At each stage, the team should keep a clear code version, record meaningful changes, and test affected workflows.

The project team consists of Agnel Faria (2605), Vaishnavi Sharma (2666), and Amlisha Cardoso (2608). The supplied project notes do not assign module ownership to individuals, so no work is attributed to a specific student here. Complete the matrix below using the tasks each member actually performed.

Team member	Actual work completed	Evidence to attach
Agnel Faria (2605)	[Complete from team records]	[Code, commits, screenshots, notes]
Vaishnavi Sharma (2666)	[Complete from team records]	[Code, commits, screenshots, notes]
Amlisha Cardoso (2608)	[Complete from team records]	[Code, commits, screenshots, notes]

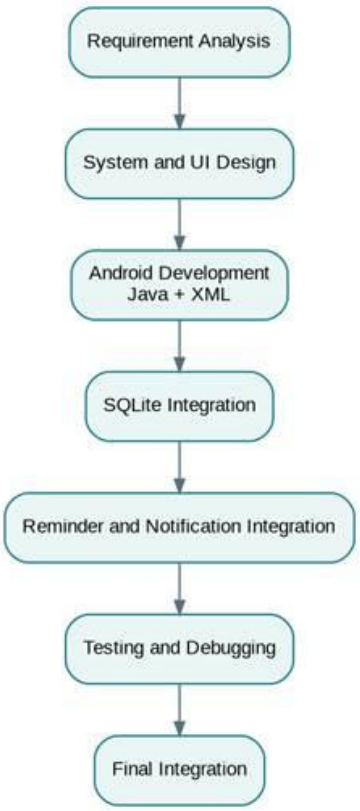


Figure 2.1: Development lifecycle described in the project notes.

Chapter 3: Requirement Analysis

Requirement analysis translates a general problem into specific behaviours the application should provide. The supplied project notes identify authentication, profile management, medicine records, medical report details, emergency contacts, medicine and appointment reminders, Problem Resolver suggestions, MediBot responses, account management, and separation of data between registered users.

A clear requirement has an observable result. For example, “support medicine management” is broad. A testable requirement states that a logged-in user can add a medicine, see it in a list, edit its details, and delete it, while the list is filtered for the current user. Testable requirements make it easier to decide when a module is complete.

This chapter separates functional requirements, which describe what the software does, from non-functional requirements, which describe qualities such as usability, availability, reliability, and maintainability. It also identifies the main user, use cases, and feasibility considerations.

3.1 Stakeholders and User Needs

The primary stakeholder is the individual using MediSync to maintain personal health-related information. The user needs to create or access an account, navigate between modules, enter details without excessive typing, locate saved information, and understand how reminders and general suggestions work. The app should also provide clear feedback when required fields are missing or login details are invalid.

The project team is another stakeholder because the application must remain manageable to develop and test within the mini-project scope. Modular features and a defined local data model make changes easier to manage. Assessors may evaluate whether the application demonstrates the stated functionality and whether the report accurately describes what was implemented.

The supplied description does not include separate doctor, clinic, administrator, or caregiver accounts. Those roles should not appear as current system actors unless the final source code shows they were implemented. The current user is an individual account holder who manages their own records.

Safety-related clarity is also a user need. Problem Resolver and MediBot provide general information only. The application should not imply that

suggestions are personalised medical treatment, and users should seek qualified professional support for serious or persistent concerns.

3.2 Functional Requirements

The following functional requirements are derived from the supplied MediSync project document. The IDs are added for report readability and can be used in the testing matrix.

ID	Requirement	Expected observable behaviour
FR-01	Registration	A user can enter account details and create a local account.
FR-02	Login / logout	Credentials are checked and a user can end the session.
FR-03	Remember Me	Selected login state is preserved when this option is enabled, as implemented.
FR-04	Health profile	User can save and update age, blood group, allergies, and notes.
FR-05	Medicine records	User can add, view, edit, and delete medicine details.
FR-06	Medical report details	User can maintain title, doctor, hospital, date, and notes.
FR-07	Emergency contacts	User can save and manage contact name, relationship, and phone number.
FR-08	Medicine reminders	User can select a reminder date/time and schedule a notification.
FR-09	Appointment reminders	User can save appointment details and schedule a reminder.
FR-10	Problem Resolver	User can select a common problem and read predefined suggestions.
FR-11	MediBot	User can submit supported questions and access prepared responses.
FR-12	Emergency dialer	App opens the dialer with the saved number for user confirmation.
FR-13	Account settings	User can view account details, change password, adjust reminder settings, and log out.
FR-14	Data separation	Queries retrieve records associated with the logged-in user ID.

3.3 Non-Functional Requirements

Non-functional requirements describe the quality and constraints of the application. The items below are based on the project notes and intended use and should be assessed through device testing and source review.

ID	Quality	Requirement / interpretation
NFR-01	Usability	Screens should be understandable and provide consistent navigation.
NFR-02	Offline use	Core record functions should work with local storage without continuous connectivity.
NFR-03	Efficiency	Local queries should support ordinary record retrieval and updates.
NFR-04	Validation	Invalid or empty input should receive meaningful feedback.
NFR-05	Maintainability	Modules should be organised so changes can be tested in isolation.
NFR-06	Data separation	Record display and operations should be scoped to the current user ID.
NFR-07	Notifications	Reminder behaviour should be checked on the target Android version.
NFR-08	Privacy	The report should state current security limitations and not claim absent encryption.
NFR-09	Guidance clarity	MediBot and Problem Resolver provide general information, not diagnosis or treatment.

3.4 Inputs, Processing, and Outputs

Inputs include account credentials, health profile details, medicine information, report metadata, emergency contacts, appointment information, selected Problem Resolver categories, and MediBot questions. Reminder date and time selection is important because invalid or past values can produce confusing behaviour.

Processing includes input validation, credential checking, SQLite operations, retrieval of records for the current user ID, storage of selected preferences, alarm scheduling, handling triggered events, and selecting prepared text responses. The process should keep the active account context consistent across modules.

Outputs include confirmation or error messages, record lists, profile information, suggestions, MediBot responses, text-to-speech output, and Android notifications. Opening Emergency results in a dialer screen with a saved number, not an automatic call. Each test should trace an input through the application to its visible output.

Input	Processing	Output
Medicine details	Validate and save under current user ID	Saved medicine in list

Appointment date/time	Save details and schedule alarm	Saved appointment and later notification
Selected everyday problem	Find matching predefined text	Suggestion displayed
Saved emergency number	Create dialer Intent	Dialer opens with number
Supported MediBot question	Match to prepared response	Answer and optional spoken output

3.5 Main Actor and Use Cases

The primary actor is the registered user. The main use cases are account access, profile management, medicine and report management, contact storage, reminder scheduling, viewing predefined suggestions, using MediBot, and opening the emergency dialer. The current project description does not define a separate clinician or administrator actor.

Use cases connect requirements to user-visible outcomes. “Manage medicine records” includes creating, reading, updating, and deleting entries. “Schedule reminder” includes selecting a date and time and requesting an Android alarm. “Use emergency contact” means retrieving the saved number and presenting the dialer. “Use MediBot” means receiving a prepared response for supported input, not a free-form AI diagnosis.

Alternate paths are also important: invalid login, missing required fields, no emergency contact saved, unsupported MediBot question, or notification permission unavailable. These should be considered in testing and reflected in the final diagram if the application implements the paths.

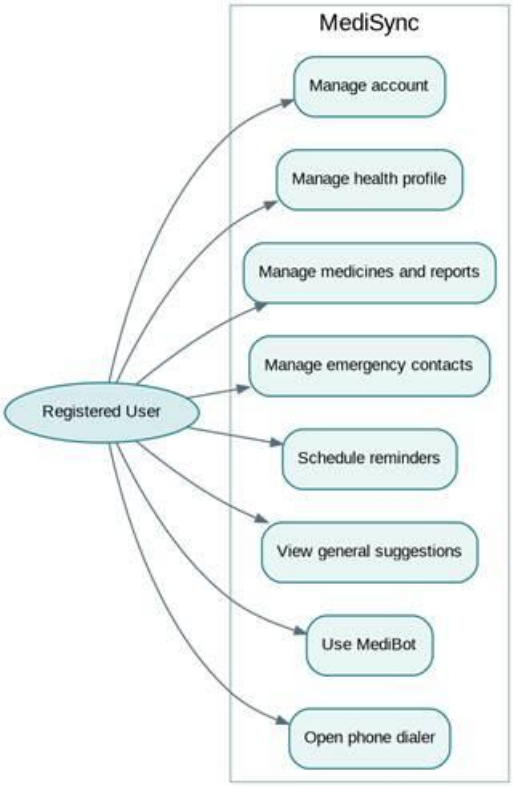


Figure 4.3: Use case diagram, included here as the requirements-level interaction model.

3.6 Feasibility and Constraints

Technical feasibility: Java, XML, Android Studio, SQLite, and the Android APIs identified in the notes are sufficient for the described local functions. Local storage avoids the need for a backend in the initial version. Reminder and dialer behaviour still depends on Android version and device configuration.

Operational feasibility: familiar forms, lists, selection controls, date/time pickers, and a dashboard can make the app understandable. Actual usability should be verified against the final screens and, where possible, user feedback.

Schedule feasibility: modular development and reducing MediBot to prepared responses keeps the project manageable. Deferring cloud backup and clinic workflows leaves more time for the current features to be implemented and tested.

Security constraint: the supplied notes state that database encryption and secure cloud authentication have not been implemented. The application should therefore be treated as a learning prototype rather than a production-ready system for highly sensitive clinical records.

3.7 Requirement Traceability

Requirement traceability connects each requirement to the module that implements it, the relevant source file or method, the test that verifies it, and the screenshot that provides evidence. It reduces the chance that a feature appears in the report but is absent from the app.

For example, FR-05 should map to the Medicine portion of Record Management, the SQLite insert/select/update/delete operations, and tests for adding, viewing, editing, and deleting a medicine. FR-08 should map to medicine reminder scheduling, the alarm and receiver implementation, notification permission checks, and a test performed at a suitable future time.

The exact source filenames and method signatures are not included in the supplied summary document. Those details should be populated from the final code rather than guessed. Traceability is particularly useful when inserting screenshots because each image can be labelled with the requirement or test case it supports.

Requirement group	Main module	Potential evidence
FR-01 to FR-03	Authentication / Account	Registration, login, invalid login, logout
FR-04 to FR-07	Record Management	Profile, medicine, report, contact CRUD
FR-08 to FR-09	Reminder Management	Reminder form, saved event, notification
FR-10 to FR-11	Resolve / MediBot	Problem selection, answer, voice/TTS
FR-12	Emergency	Dialer opens with saved number
FR-14	Multi-user data management	Two accounts with distinct test records

Chapter 4: System Design

System design defines how the user interface, application logic, Android services, and local storage work together. MediSync uses XML layouts and Java logic. SQLite stores structured records, while SharedPreferences maintains login state and selected settings. Reminder events use AlarmManager and a BroadcastReceiver, and the emergency action uses an Intent to open the phone dialer.

The design goal is to keep the user journey simple while supporting distinct features. The dashboard provides entry points for Record, Remind, and Resolve. MediBot, Emergency, and Account Management are also accessible. Data-related modules use the logged-in user ID so records can be retrieved for the appropriate account.

The diagrams in this chapter are logical models prepared from the supplied project description. They illustrate relationships and behaviour but do not claim exact source-level class names. Revise them if the final implementation differs

4.1 High-Level Architecture

The architecture can be understood in layers. The presentation layer consists of Android screens laid out using XML and connected to Java event handlers. The application logic layer validates input, selects the required operation, and coordinates navigation. The persistence layer uses SQLite for structured records and SharedPreferences for selected local preferences or session state.

Some tasks use platform services. AlarmManager schedules a future event. A BroadcastReceiver reacts when a scheduled event is delivered and can display a notification. An Android Intent requests that the operating system open the phone dialer with the saved emergency number. Speech-related interaction allows supported MediBot questions to be entered by voice and responses to be read aloud.

The layers are connected but conceptually distinct: a screen captures input, code validates it, and a database operation stores or retrieves it. Figure 4.1 summarises these relationships. Compare this overview with the source code before final submission.

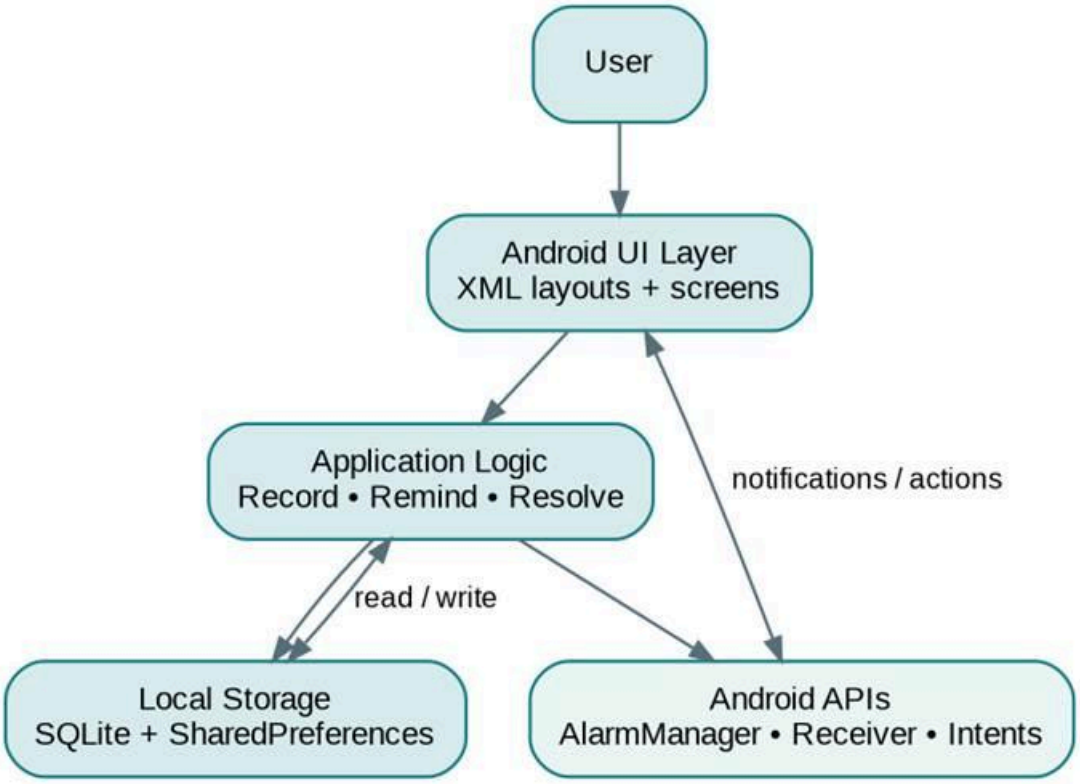


Figure 4.1: Logical architecture of MediSync.

4.2 Application Navigation and Screen Flow

The normal flow begins when the application opens, continues to login or registration, and reaches the dashboard after successful authentication. From the dashboard, the user chooses a main module or opens Account Management.

Record groups the health profile, medicine records, report details, and emergency contacts. Remind contains medicine and appointment reminder workflows. Resolve contains the selectable Problem Resolver. MediBot provides its question-and-answer interface. Emergency opens the dialer based on saved contact information. Account Management provides account details, password change, preferences, and logout.

Screens should offer predictable back navigation and feedback after save, update, or delete operations. Empty states should explain what the user can do next rather than display a blank list. Figure 4.2 is a module-level navigation map and should be supplemented with actual screen screenshots.

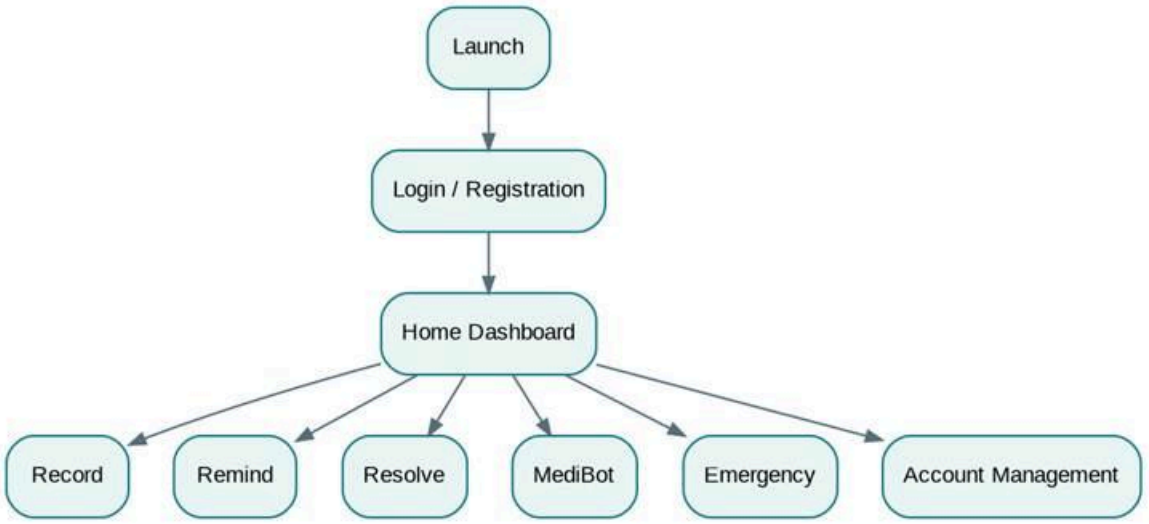


Figure 4.2: Application navigation flow (logical overview).

4.3 Authentication Activity Flow

Authentication starts when the user opens the application and selects login or registration. The user enters account information, and the application validates the input. During login, the supplied username and password are checked against stored credentials. Invalid details should produce an error and allow a retry; valid details lead to the dashboard.

Registration similarly requires validation before a new account record is created. Remember Me affects whether login state is preserved according to the implementation. SharedPreferences is used for login state and selected preferences, as described in the project notes.

After login, the account identity must be available to data modules. The user ID should be used when accessing the profile, medicines, reports, contacts, and appointments. On logout, appropriate session state should be cleared so another user does not inherit the previous account's visible context.

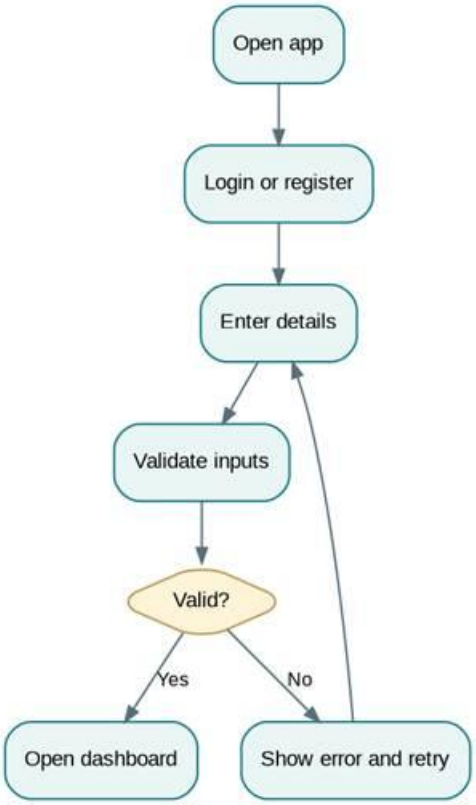


Figure 4.4: Logical authentication activity flow.

4.4 Record Management Activity Flow

The Record workflow starts with selecting the information type: health profile, medicine, medical report, or emergency contact. The user enters details using the relevant form, after which required fields and basic formats should be validated. The application performs the selected SQLite operation for the currently logged-in user.

A new entry creates a record. An existing entry can be retrieved, displayed for editing, and saved again. Deletion should target the selected record and provide feedback. The refreshed list should reflect the database state rather than leave an outdated item on screen.

Although fields differ by record type, the same general pattern applies: validate, scope to user ID, execute the operation, and display the result. Figure 4.5 represents this flow at a logical level; update it if the final screen sequence differs.

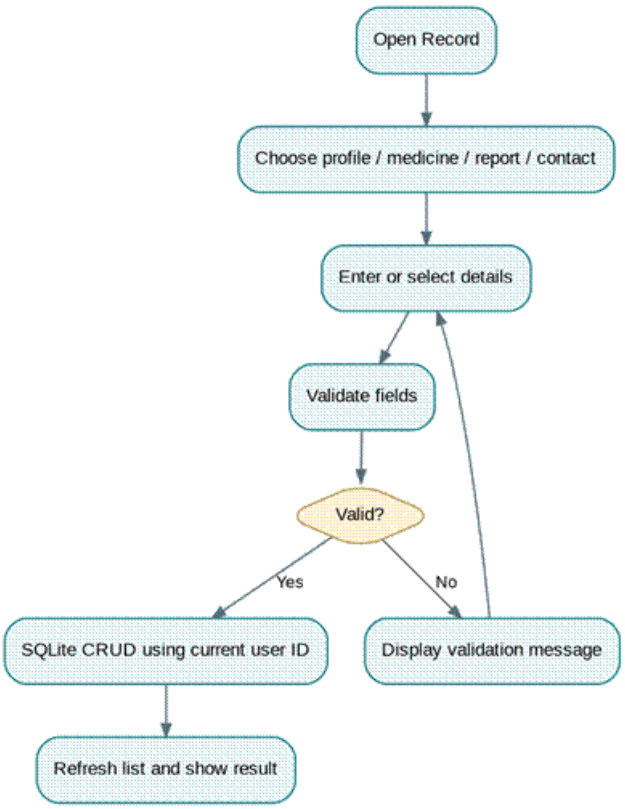


Figure 4.5: Logical activity flow for record operations.

4.5 Reminder Sequence Design

A reminder involves more than saving text. The user enters a medicine or appointment event and chooses a date and time. The application stores the details and requests a scheduled alarm. When the event is delivered, a BroadcastReceiver handles it and can display a notification.

The saved record and scheduled alarm are related but distinct. Editing or deleting an event should be checked to ensure that an old alarm is updated or cancelled as appropriate. A unique identifier can help distinguish events; the project notes state that user-specific identifiers are used for scheduled reminders.

Notification behaviour may be affected by Android version, permission settings, battery restrictions, and device configuration. A test that only confirms the event was saved does not prove that the notification appears. Scheduling and notification display should be tested separately.

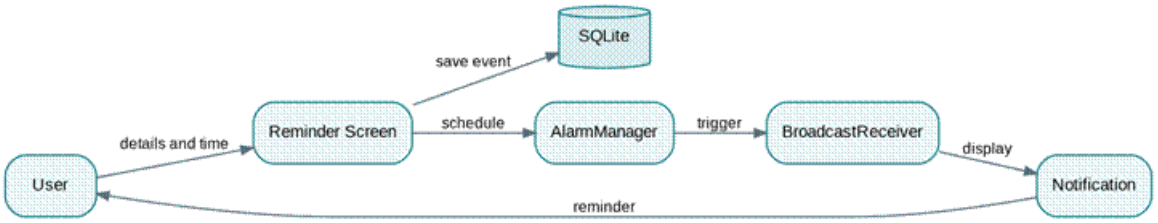


Figure 4.6: Conceptual reminder-notification sequence.

4.6 Database Design and ER Diagram

SQLite stores the six main record groups described in the project notes: users, health profiles, medicines, medical reports, emergency contacts, and appointments. Health-related records are linked to a unique user ID so the current account can retrieve its own information.

The logical model gives the users table a unique identifier and associates each other record type with that identifier. Exact table names, column names, constraints, indexes, and types must be checked against the implemented DatabaseHelper schema. The diagram is a readable conceptual overview, not a claim that the displayed identifiers exactly match the SQL source.

The project notes confirm user-specific filtering but do not provide the complete schema. If the assessment requires an exact ERD, prepare it from the actual SQLite definitions and replace any field or relationship that does not match the code.

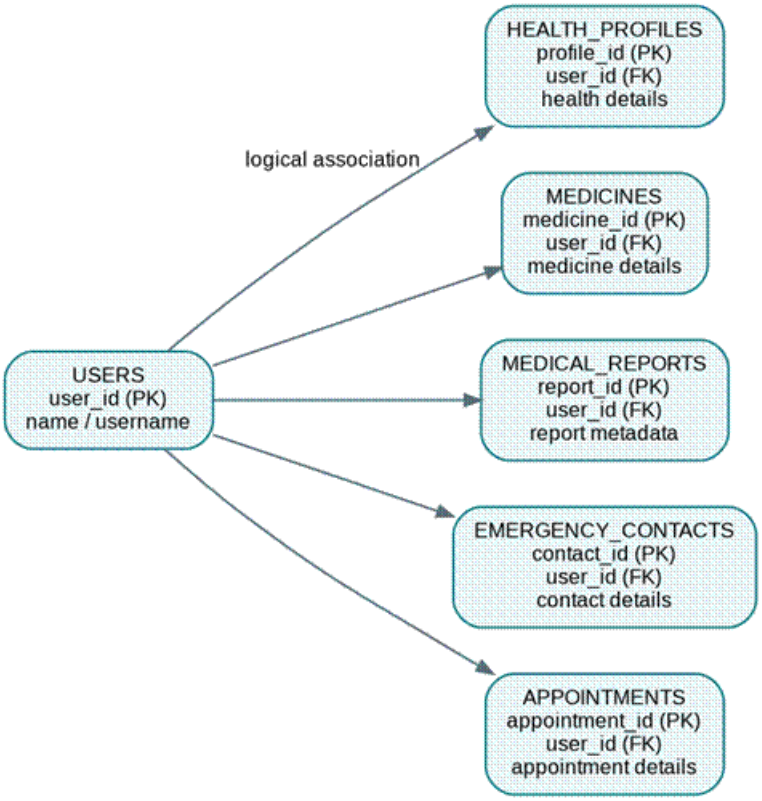


Figure 4.7: Logical data model derived from the supplied project description.

4.7 Logical Component / Class View

The implementation can be explained as collaborating components: account access, dashboard navigation, record management, reminder management, Problem Resolver, MediBot, emergency contact access, local database operations, preferences, and Android platform APIs.

A true UML class diagram should show the actual classes, attributes, methods, and relationships in the Java code. The supplied document identifies a DatabaseHelper class but does not list every source file or method signature. Figure 4.8 is therefore a logical component view, not a verified code-level class diagram.

For the final version, inspect the project tree and show only classes that truly exist. Include actual activities, database helpers, model classes if present, reminder receiver classes, and interfaces or enums only when they are implemented.

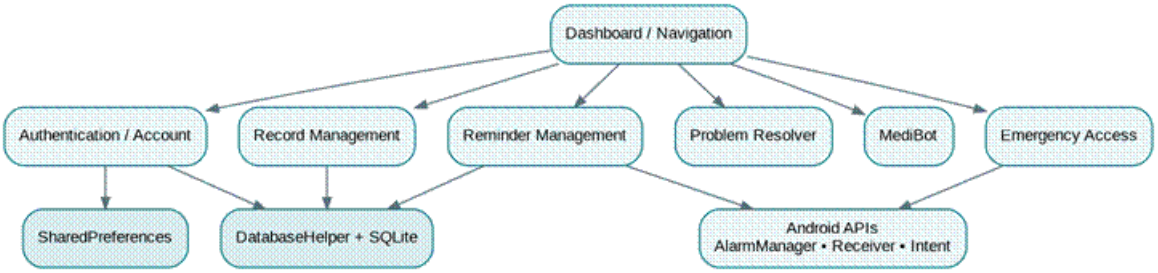


Figure 4.8: Logical component view; validate against actual Java classes.

4.8 CRUD and Data Flow

CRUD means Create, Read, Update, and Delete. MediSync supports these basic operations for its main record types. When the user provides input, the application should validate values, execute the database operation, refresh the visible data, and display a result. Operations involving personal records should use the logged-in user ID.

Creation inserts a row. Reading retrieves stored rows or one selected item. Updating changes an existing record. Deleting removes the selected record. The interface should not show success if the database operation fails. Queries must scope record retrieval and modification to the appropriate user.

Database testing should use more than one account to confirm that the filtering behaviour is consistent across profiles, medicines, reports, contacts, and appointments. Figure 4.9 shows the simplified flow from a feature screen through validation to SQLite and back to the interface.

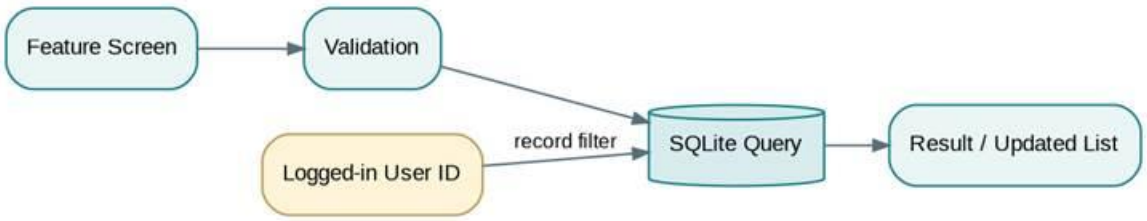


Figure 4.9: Simplified CRUD data flow.

4.9 User Interface Design Principles

The project notes state that the interface uses buttons, selectable options, date pickers, and time pickers to reduce unnecessary typing. These controls suit a personal organiser because many inputs follow known formats. A time picker, for example, can reduce inconsistent time entry.

Consistency matters across the app. Similar forms should use similar labels and button placement. Save, update, and delete actions should be distinguishable.

Navigation should make it possible to return to the dashboard without losing data. Empty fields should have meaningful labels, and error messages should explain what needs correction.

Accessibility should be considered even in a mini project. Text should be readable, touch targets usable, and important feedback should not depend on colour alone. Voice input and text-to-speech can help some users, but may vary with device and environment. The final report should use screenshots of the actual interface rather than mockups.

Chapter 5: Tools and Technologies

MediSync is described as an Android application developed in Android Studio. Java implements application logic, XML defines interface layouts, SQLite provides local structured storage, and Android APIs support reminders, notifications, dialer access, and speech-related interaction. SharedPreferences is used for login state and selected settings.

The choices match an offline-oriented native Android mini project. Java handles control flow and event responses, XML separates layout description from program logic, SQLite stores information across sessions, and platform components integrate with device behaviour. This chapter explains the purpose of each technology without claiming additional services or libraries that are not described in the project notes.

5.1 Android Studio

Android Studio is the development environment used to build and run the application. A typical Android project organises Java source files, XML layouts, manifest declarations, resources, build configuration, and debugging tools. It supports launching an application on a device or emulator and inspecting runtime output.

For MediSync, the environment supports development of the dashboard, forms, lists, reminder screens, and account interfaces. The team can test navigation, validation, database operations, and notification behaviour. Device testing is important for scheduled alarms because emulator and hardware settings may differ.

When a defect appears, the team should record how to reproduce it and relevant output, correct the cause, and rerun the affected scenario. Screenshots should

not reveal real passwords, API keys, or health information. Demonstration accounts should use dummy data created for testing.

5.2 Java Programming Language

Java implements the application logic. In an Android app, Java code responds to user actions, validates forms, performs database operations, schedules reminder events, navigates between screens, and coordinates platform features. The language supports object-oriented organisation through classes, methods, encapsulation, and reusable components.

A module such as Medicine Management may need operations for saving a medicine, retrieving a list, updating an entry, and deleting a record. Authentication checks user input and manages session information. Reminder logic converts selected date and time values into a scheduled event request. Exact class and method names vary by implementation and should be documented from the source code.

A database helper can centralise schema creation and queries while UI classes handle events and rendering. Clear boundaries make testing and future maintenance easier. The final report should reference the actual classes rather than generic examples where exact source code is available.

5.3 XML Layouts and Android Resources

XML defines the visual layout of Android screens. Layout XML describes interface elements and their arrangement, such as text fields, buttons, labels, and containers. Java code accesses those views, responds to button clicks, validates values, and updates the interface.

Separating layout description from event logic improves readability. A layout defines how a screen looks; Java defines what happens when the user interacts with it. Shared colours, strings, dimensions, and drawable resources may also be stored in resource files to maintain consistency.

For MediSync, XML-based UI design supports authentication, the dashboard, Record forms, reminder screens, Problem Resolver, MediBot, Emergency, and Account Management. The exact number and names of layout files should be stated only after checking the project directory. Actual screenshots should be captioned and linked to the feature they demonstrate.

5.4 SQLite and DatabaseHelper

SQLite is the local relational database used by MediSync. The project notes describe tables for users, health profiles, medicines, medical reports, emergency contacts, and appointments. This structure supports saving records locally and retrieving them later without a continuous internet connection.

A DatabaseHelper class manages database creation, updates, and operations. CRUD methods insert new records, retrieve existing entries, update values, and delete records. The logged-in user's unique ID is used to associate records with an account and filter retrieved data.

SQLite suits a local prototype, but local storage does not automatically synchronise across devices. Access after loss or uninstall may depend on backup and device behaviour. The project notes also state that database encryption has not been implemented. The final report should include exact table definitions and column names from the source when available; the ER diagram in Chapter 4 is conceptual.

5.5 SharedPreferences and Session State

SharedPreferences stores small key-value preferences locally. MediSync uses it to maintain login state and selected user-specific settings, according to the supplied project notes. It is not a replacement for SQLite when managing lists of related records such as medicines or appointments.

After successful login, the application needs a reliable way to identify the current user. When the user logs out, session-related values should be cleared or updated appropriately. If Remember Me is enabled, the app may preserve login state as implemented; the exact behaviour should be verified in the final build. Preferences may also include reminder settings if those are implemented in the source.

SharedPreferences should not be treated as a secure credential vault. The project notes identify security as an area requiring improvement, including stronger password protection and additional safeguards before any wider deployment.

5.6 AlarmManager, BroadcastReceiver, and Notifications

The Reminder feature uses AlarmManager to schedule medicine and appointment events. Selected date and time values are converted into an alarm request. A BroadcastReceiver handles the delivered event and displays a

notification. The project notes also mention notification-permission handling on supported Android versions.

The behaviour must be tested end-to-end. A successful save operation demonstrates that a record was stored; it does not prove that an alarm was scheduled correctly or that Android displayed the notification. Separate tests should check a valid future time, permission state, editing or deleting reminders, and disabled reminder settings.

Alarm and notification behaviour can vary across Android versions, battery restrictions, and device manufacturers. The report should record the target device and observed behaviour rather than promise guaranteed delivery. The feature is intended as an everyday prompt, not a medical-grade alert system.

5.7 Android Intents and Speech Features

Android Intents allow an app to request an action from another component or application. In MediSync, the Emergency feature uses an Intent to open the device dialer with a saved emergency number. The user can review the number and choose to call; the application does not automatically place emergency calls.

MediBot includes voice input and text-to-speech output according to the project notes. Voice input can reduce typing, while text-to-speech can read a prepared answer aloud. These capabilities depend on device support, settings, language, and the implementation.

MediBot responses are predefined rather than generated by a full generative AI model. The offline approach can provide predictable answers to supported questions, but it cannot answer every query or provide personalised clinical advice. Screenshots should illustrate the actual interface without exposing private contacts or health information.

5.8 Technology Stack Summary

The table summarises the tools and platform components identified in the project notes.

Technology	Purpose	Project use
Android Studio	Development and debugging	Build and test the Android application
Java	Application logic	Event handling, validation, data operations, navigation
XML	UI layout definition	Forms, screens, labels, and controls
SQLite	Local relational storage	Users, profiles, medicines, reports, contacts, appointments
DatabaseHelper	Database lifecycle and operations	Creation, updates, CRUD and retrieval
SharedPreferences	Local preferences/session state	Login state and selected settings
AlarmManager	Schedule timed events	Medicine and appointment reminder scheduling
BroadcastReceiver	Handle reminder events	Receives events and triggers the notification flow
Android Intent	Request platform action	Open dialer with the saved emergency number
Speech input / TTS	Alternative interaction	Voice questions and spoken prepared responses

Chapter 5 Summary

The stack matches the project's local, offline-oriented scope. Java, XML, and Android Studio form the core development environment; SQLite stores structured data; SharedPreferences stores selected preferences and session state; and Android APIs support reminders, notifications, dialing, and speech-related interaction.

The stack also defines the current limits. Local persistence does not provide automatic cloud synchronisation, standard local storage does not itself imply encryption, reminders depend on Android behaviour and permissions, and MediBot remains a prepared-response assistant. These boundaries are revisited in implementation and future scope.

Chapter 6: Implementation of Modules

The implementation phase converted the design into Android screens and feature logic. According to the project notes, development began with the dashboard and navigation, followed by Record, Reminder, Problem Resolver, MediBot, Emergency, authentication, and account-management functions. SQLite operations and Android reminder mechanisms were integrated as features became available.

This chapter describes the responsibilities and behaviours confirmed by the supplied project material. Exact source-file names, class signatures, and internal algorithms should be added only after reviewing the actual Java code.

6.1 User Authentication

The User Authentication module allows a person to register and sign in. Registration collects a name, username, and password according to the supplied notes. The application validates fields and stores account details in SQLite. During login, the entered username and password are checked against stored credentials. Successful authentication redirects the user to the dashboard.

The module also includes Remember Me, logout, account details, and password change. SharedPreferences maintains login state and selected preferences. After authentication, the logged-in username is used to identify the matching user ID, which is then used by other modules to retrieve records.

Important checks include successful registration, successful login, invalid credentials, empty fields, logout, and Remember Me behaviour. The security chapter must not claim hashing, encryption, biometrics, or secure cloud authentication unless the source code proves those features exist.

- Registration creates an account after validation.
- Login checks the supplied account details.
- Successful login opens the dashboard.
- Logout ends the active session as implemented.
- Remember Me preserves selected login state when enabled.
- Account settings expose the supported details and password change.

6.2 Record Management: Health Profile

The Health Profile stores selected personal health information, including age, blood group, allergies, and additional notes. The information can be saved and updated. The profile is associated with the current user so it can be retrieved when that user returns to the Record area.

The form should use meaningful labels and validate required values. Where fields have a known format, selection controls can reduce input errors. The interface should distinguish an empty field from a saved value and reflect the stored information after saving.

The profile is not a complete clinical history. The supplied scope does not describe automated profile analysis, medical recommendations based on allergies, or integration with external health records. Testing should include saving a profile, reopening it, changing one field, and confirming persistence. Another account should not see the first user's profile.

6.3 Record Management: Medicine Records

The Medicine section maintains medicine name, dosage, frequency, start date, end date, and reminder time, as listed in the project notes. Users can add records, view existing entries, edit information, and delete entries.

The form saves information that may be relevant to the reminder workflow. Dosage and frequency are descriptive details entered by the user; the app should not be described as selecting or prescribing medication. A reminder is a scheduled prompt and does not validate whether a medicine is clinically appropriate.

The main implementation concern is keeping the visible list consistent with the database. After an edit or delete operation, the list should refresh. When accounts change, the list should use the active user ID. Tests should cover valid input, missing fields, date handling, editing, deletion, persistence, and data separation.

6.4 Record Management: Medical Report Details

The Medical Reports section stores information about a report rather than operating as a complete document library. The supplied notes list report title, doctor name, hospital name, report date, and additional notes.

This metadata can help a user remember which report exists and when it was created, but it does not mean the app stores the report itself as a PDF or image. Document upload is listed as future scope. The final report must maintain this distinction so that readers do not expect an upload or viewer that is absent from the current build.

Core behaviours are saving the details, retrieving and displaying them, allowing corrections, and deleting the record. The operation should be linked to the current user ID. Testing evidence should use fictional examples rather than real patient or report information.

6.5 Record Management: Emergency Contacts

The Emergency Contact section saves contact name, relationship, and phone number. It stores the details needed by the Emergency action. The Record module maintains the contact information; the Emergency module retrieves the saved number and launches the dialer. According to the project notes, the user manually places the call.

The screen should let the user correct an incorrectly entered number and provide appropriate feedback for missing required values. If multiple contacts can be stored, the final code and interface determine how the emergency contact is selected. That behaviour should be explained using the actual implementation, not assumed from the logical design.

Tests should cover saving a contact, reopening it, editing the number, deleting it, and using the Emergency shortcut when a contact exists and when none has been configured.

6.6 Reminder Management: Medicine Reminders

Medicine reminders let users select a medicine and reminder date/time. The app uses AlarmManager to schedule an alarm. When the event is delivered, a BroadcastReceiver handles it and displays a notification. The notes mention identifiers to distinguish reminders and notification permission handling on supported Android versions.

The saved medicine record and scheduled event must remain consistent. The project should test whether reminders are updated or cancelled when a related entry is changed or deleted, according to the intended implementation. The notification should be understandable and should avoid exposing unnecessary sensitive detail on a lock screen.

Tests require a suitable future time and a device with notification settings in a known state. Test permission denial, past times, edits, deletions, and disabled reminder settings. After screenshots arrive, include the medicine reminder form, a saved event, and an actual notification if available.

6.7 Reminder Management: Appointment Reminders

Appointment reminders support upcoming medical appointments. The fields described in the project notes include appointment title, doctor name, hospital name, appointment date, appointment time, and notes. The app stores appointment details in SQLite and schedules notifications using Android alarm functionality.

Users can view, edit, and delete saved appointments. This makes the feature both a record-management workflow and a timed event workflow. A correct implementation should retain the appointment information in the list and associate the reminder with the appropriate event.

Useful tests include creating a future appointment, reopening it, editing its time, deleting it, and checking how the scheduled notification responds. The appointment list should also be filtered by the current user ID. MediSync records user-entered appointment information; the supplied notes do not describe booking directly with a hospital or synchronising with a doctor’s calendar.

6.8 Problem Resolver

Problem Resolver provides general suggestions for common everyday problems. The user chooses a category and then a particular problem. The application displays predefined suggestions associated with that selection. This selectable flow reduces the need to type a long description.

The module is intentionally limited. It does not diagnose medical conditions, prescribe medicines, or provide personalised treatment. It can respond only to the categories and problems included in the application. This boundary should be maintained throughout the report and demonstration.

Testing should verify that each available category leads to the expected choices and that selecting a problem displays the matching suggestion. The application should also handle missing selections clearly. A pair of screenshots—category selection and displayed suggestion—would provide useful evidence.

6.9 MediBot

MediBot is an offline health-information assistant with a chat-based interface. The project notes describe text input, voice input, prepared offline responses, suggested questions, and text-to-speech output. Its responses are predefined rather than generated by a generative AI model.

The offline approach can provide predictable responses to supported questions without continuous network access. However, it cannot cover every query, understand every wording variation, or provide personalised medical advice. Unsupported questions should be handled clearly instead of producing an unrelated answer that appears authoritative.

Testing should confirm typed questions, suggested questions, voice entry if enabled, response matching, and spoken output where available. MediBot is a general-information feature, not a substitute for professional diagnosis or treatment. The final report should name the sample questions supported by the current response set and avoid claiming AI capabilities that have not been implemented.

6.10 Emergency and Account Management

The Emergency module retrieves saved contact information and opens the device phone dialer with the saved number. The user reviews the number and manually places a call. If no emergency contact is available, the app directs the user to the Emergency Contact screen. It does not automatically call emergency services.

Account Management supports viewing account details, changing a password, viewing application information, enabling or disabling reminder settings, and logging out. SharedPreferences is used to maintain login information and selected preferences according to the project notes.

Logging out should prevent the previous account’s data from appearing as though it belongs to the next user. The exact password rules and credential storage mechanism are not included in the supplied summary; the report should not claim strong password policies or secure credential storage without

source-level evidence. Testing should include account details, password change, logout, Remember Me, reminder settings, and both Emergency paths.

6.11 Multi-User Data Management and Integration

MediSync supports multiple registered users on one device. Each account receives a unique user ID, and database operations use that ID to retrieve relevant health information. The project notes state that profiles, medicines, reports, emergency contacts, and appointments are filtered for the current account.

Data separation depends on using the correct user ID consistently in read, update, and delete operations. If a query omits the user filter, one account might see or modify another user's records. Testing should use two accounts with distinct dummy entries and repeat the check for each record type rather than only medicine lists.

Final integration connects modules through the dashboard, account identity, database, and Android platform. Check that registration leads to the dashboard, records remain visible after saving, reminders use the selected date and time, the dialer uses the saved contact, and logout changes the active session. User-specific filtering is helpful but is not equivalent to encryption or comprehensive security.

Chapter 7: Testing and Validation

Testing verifies whether MediSync's modules behave as expected. The supplied notes list checks for registration, login, input validation, record operations, reminders, Problem Resolver, MediBot, Emergency, account management, navigation, and separation of records between users.

The source notes include 15 functional test cases marked Pass, but they also recommend confirming the results against the final installed application. Before submission, rerun the tests, update statuses if needed, and attach evidence for the important cases. A screen opening successfully is not enough to prove reminder delivery or data separation.

7.1 Testing Objectives and Process

Testing objectives are to verify registration and login, required-field validation, database operations, reminder scheduling, Resolver and MediBot output, Emergency dialer behaviour, and separation between user accounts.

Each test case should state preconditions, test data, steps, expected result, actual result, status, date, and evidence. Dummy accounts and non-sensitive sample information make screenshots easier to share. When a defect is found, record reproduction steps, correct the cause, and rerun both the failed test and nearby workflows.

Figure 7.1 shows a simple test cycle. The final report should include a test execution date, device model, Android version, and application version after the group completes its final review.

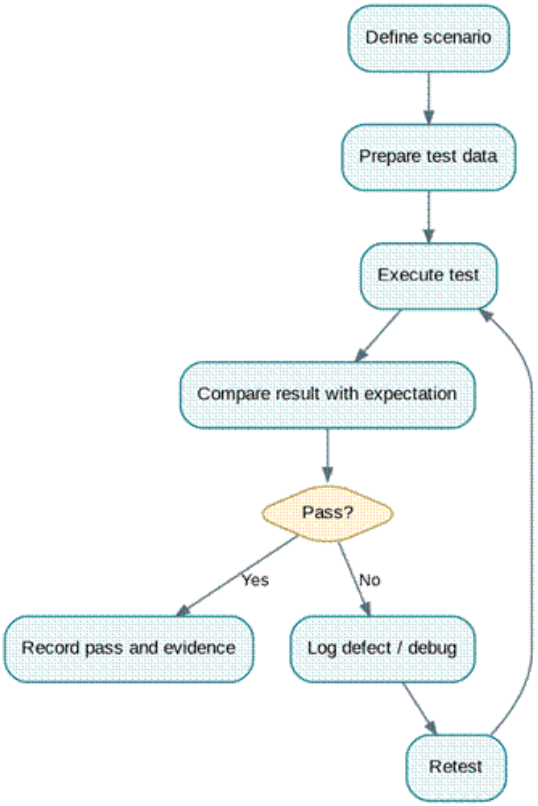


Figure 7.1: Testing and debugging cycle.

7.2 Functional Test Register: Authentication and Records

The following cases are organised from the project test register. Statuses reflect the supplied notes and should be rechecked against the final build.

ID	Test case	Expected result	Reported status
TC01	Registration	New account is created	Pass*
TC02	Valid login	Dashboard opens	Pass*
TC03	Invalid login	Error message is shown	Pass*
TC04	Health profile	Details are saved and displayed	Pass*
TC05	Medicine management	Medicine can be added, edited, deleted	Pass*
TC06	Report management	Report details are saved	Pass*
TC14	Input validation	Invalid or empty input receives feedback	Pass*
TC15	Remember Me	Selected login state is preserved	Pass*

7.3 Functional Test Register: Reminders and Other Modules

The following cases cover the remaining major functions in the source test register. Reminder behaviour should be reconfirmed on the target device.

ID	Test case	Expected result	Reported status
TC07	Medicine reminders	Notification appears at scheduled time	Pass*
TC08	Appointment reminders	Appointment saved and notification appears	Pass*
TC09	Problem Resolver	Relevant predefined suggestions displayed	Pass*
TC10	MediBot	Prepared responses work for supported text/voice features	Pass*
TC11	Emergency	Dialer opens with saved emergency number	Pass*
TC12	Multi-user separation	Each user accesses their own records	Pass*
TC13	Account management	Password change and logout work	Pass*

7.4 Database Testing

Database testing checks whether information is stored accurately and can be retrieved, updated, and deleted. SQLite stores accounts, health profiles, medicines, report details, emergency contacts, and appointments. DatabaseHelper is described as managing database creation, updates, and operations.

Insert tests check whether valid forms create records. Read tests compare the displayed values with those saved. Update tests change fields and confirm persistence. Delete tests verify the entry no longer appears. Invalid-input tests check that incomplete information is not silently accepted where fields are required.

User filtering is an important database test. Sign in as User A and create a medicine and appointment. Log out and sign in as User B. The same lists should not display A’s entries. Then create separate entries for B and return to A to

confirm that A's data remains available. Exact schema evidence can be added from the source code if required.

- Confirm required fields before insert.
- Verify values after retrieval.
- Confirm updates persist after reopening the module.
- Confirm deleted records no longer appear.
- Check current user ID filters in each relevant query.
- Use dummy values rather than real health information.

7.5 Multi-User Testing Procedure

The supplied notes describe this procedure: log in as User A and add medicine, report, and appointment information; log out and sign in as User B; open Record and Remind; verify A's entries are not displayed; save separate information for B; and return to A to confirm A's records are still available.

The reported result is that two-account testing confirmed the records remained separate. The source notes conclude that the feature worked as expected during the performed tests. This should be supported with screenshots from the final build before submission.

The test is stronger when repeated for every record type, because a correct medicine query does not guarantee that report, contact, profile, and appointment queries use the same filter. Distinct dummy entries make accidental mixing easy to identify. User-ID filtering is not the same as database encryption or protection against direct access to application files.

Step	Action	Expected result
1	Log in as A and add dummy records	A's records appear for A
2	Log out and log in as B	B sees only B's records or empty lists
3	Create distinct records for B	B's records appear for B
4	Return to A	A's original records remain; B's are hidden
5	Repeat across record types	The same boundary holds for each module

7.6 Reminder and Notification Testing

Reminder testing checks the full path from date/time selection through scheduling to notification display. The project notes identify these scenarios: scheduling a medicine reminder, scheduling an appointment reminder, selecting a valid future date and time, checking notification permissions, editing a scheduled reminder, deleting a reminder, and disabling reminder settings.

A controlled test should choose a time a few minutes in the future and keep the device in a known state. Record the selected time, permission state, when the notification appears, and relevant device settings. If the alert does not appear, investigate rather than automatically marking the case as passed.

Editing and deletion deserve special attention because an alarm can exist separately from its database entry. Verify whether changing or deleting an appointment removes or updates an unwanted old alert. Device model, Android version, and notification settings should be recorded in the final evidence.

7.7 User Interface and Navigation Testing

UI testing evaluates whether the application is understandable and screens respond consistently. The supplied notes list login and registration, dashboard, Record, Reminder, Problem Resolver, MediBot, Emergency, and Account Management as areas checked for navigation and usability.

Verify that buttons open the expected screen, back navigation returns to a logical destination, labels are readable, and forms provide useful feedback. After saving a record, the visible list should update. After logout, the previous user’s data should not remain visible in the next account’s context.

Date and time pickers reduce some input errors but still need validation. Screenshots should show enough context to identify the screen and any feedback message. Each image should be numbered and referenced in the text, rather than pasted without explanation.

7.8 Testing Results and Interpretation

The supplied project register lists 15 functional test cases and states that all passed, reporting a 100 percent success rate. That figure reflects the project notes; it is not independently verified in this report. The same notes explicitly recommend confirming the results against the final installed application.

Before submission, execute each listed case on the final build and update the status to Pass, Fail, Blocked, or Not Run. A test that has not been repeated

should not be left marked passed simply because an earlier build worked. Reminder cases need evidence of notification behaviour, and multi-user testing should use separate accounts.

A pass percentage should be calculated from completed tests. If all 15 are rerun and pass, the report can state 15/15. If a case fails or cannot be run, disclose that result and describe the correction or limitation. The final summary should be backed by concrete evidence in the screenshot chapter and appendix.

7.9 Defect Handling and Regression Testing

When a defect appears, reproduce it using known steps, identify the likely area of code, correct the issue, and retest. A defect log can record the ID, affected module, severity, reproduction steps, expected and observed behaviour, correction, and retest result.

Regression testing checks whether a code change breaks a previously working feature. A database-schema change may affect both Record and Remind. A login-session change may affect every user-specific query. A reminder-ID change may affect multiple scheduled events.

The supplied notes say that errors found during development were corrected and affected features retested, but they do not provide specific defect examples. Add genuine examples if the group has them; otherwise, do not invent bugs or fixes. A short final regression checklist can cover login, saving and reopening a record, scheduling a reminder, using Resolver, asking MediBot a supported question, testing Emergency, and logging out.

Chapter 7 Summary

Testing covers account, record, reminder, guidance, emergency, and navigation workflows. The source test register records 15 cases as passed, but the notes call for confirmation on the final installed application. Confirm the results and collect evidence before treating the register as final.

The next chapter provides structured spaces for genuine screenshots from the working app. Replace each placeholder with an actual image and a brief explanation of what the image demonstrates.

Chapter 8: Screenshots of the Working Product

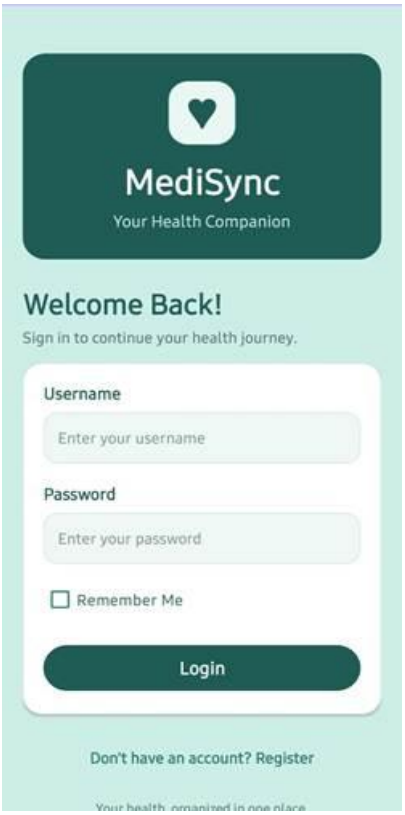
This chapter is reserved for visual evidence from the working MediSync application. Use test accounts and dummy data. Do not include real passwords, real phone numbers, personal health details, report contents, or notifications containing sensitive information. Each screenshot should have a number, a concise caption, and a sentence explaining the feature or test it supports.

8.1 Registration, Login, and Dashboard

Insert the registration and login screens, followed by a successful-login dashboard. If registration and login appear on one screen, use separate images only when they add useful information. A negative-login screenshot can show the error message, but must not reveal real credentials.

Caption the images with clear names such as “Registration screen,” “Login validation message,” and “Home dashboard after successful login.” Link the images to FR-01 and FR-02. If Remember Me is available, include evidence of its behaviour after reopening the app.







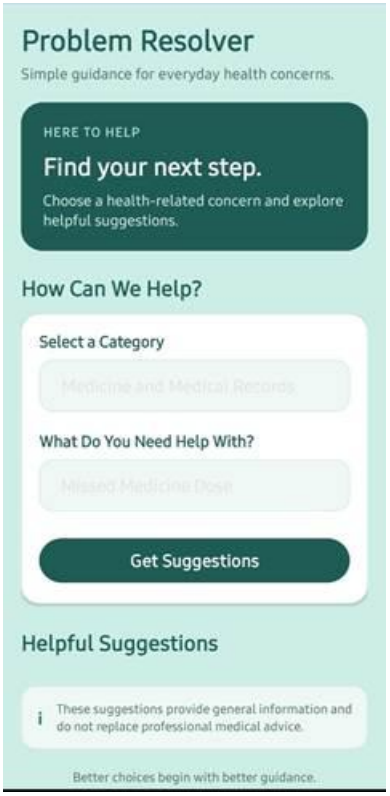
8.2 Dashboard and Module Navigation

Insert a full screenshot of the dashboard showing access to Record, Remind, Resolve, MediBot, Emergency, and Account Management where available. Explain how the dashboard groups the main features and what happens when the user selects each entry.

If the real dashboard differs from the logical navigation diagram, update the diagram to match. Use actual app screenshots rather than decorative mockups that could be mistaken for the implemented product. If the dashboard scrolls, capture separate views and label them clearly.





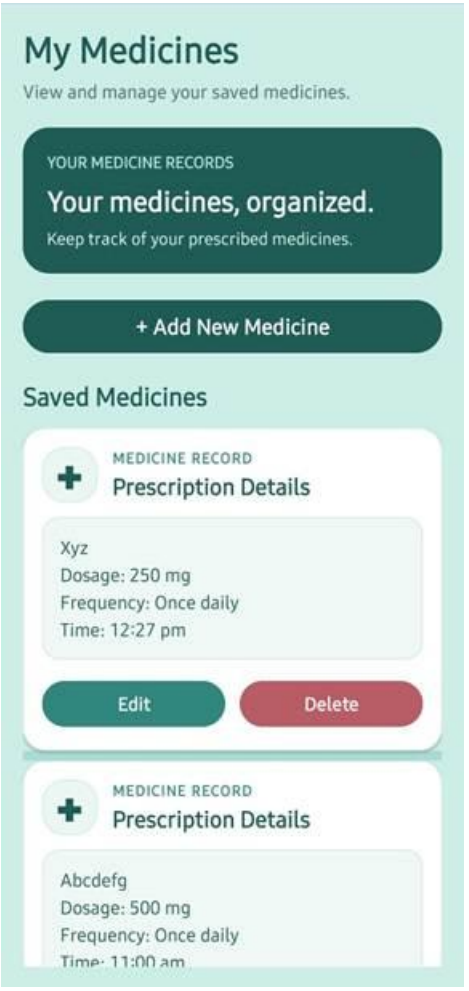


8.3 Record Screens: Health Profile and Medicine

Insert the health profile form and a saved-profile screen using dummy values. The medicine screenshots should show the fields actually present, such as medicine name, dosage, frequency, dates, and reminder time if shown. Include a medicine list after saving an entry.

If practical, add an edit or delete screenshot to demonstrate CRUD functionality. Use clearly fictional test data. Explain how the user saves or updates an entry and connect the images to FR-04 and FR-05. A screenshot proves that a screen displays information; persistence should also be checked by reopening the record.

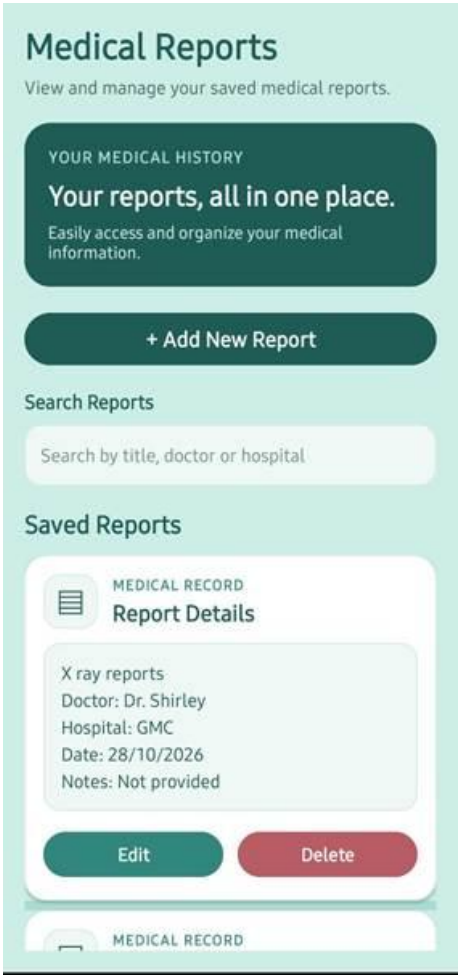




8.4 Record Screens: Reports and Emergency Contacts

Insert a screenshot of the Medical Reports form and a saved sample report. The current application stores report metadata rather than complete medical documents; do not imply that a PDF or image is uploaded unless that feature is actually present.

Insert the Emergency Contact form and saved contact list if available. Use a fictional name and dummy number. Explain which fields are required, how the information is saved, and how the record is retrieved later. Link the images to FR-06 and FR-07.

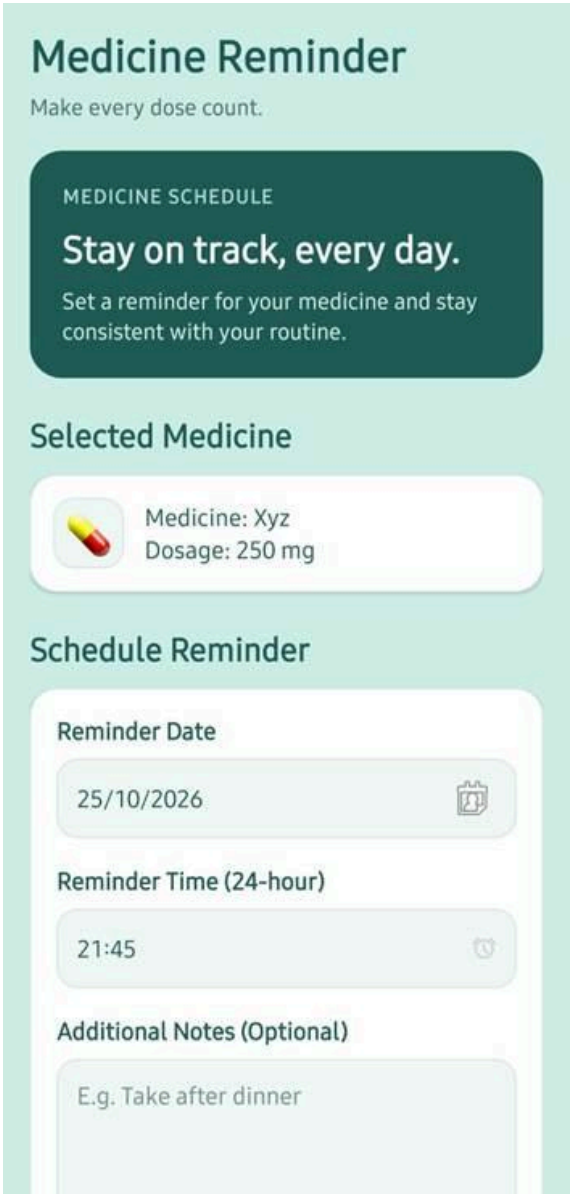


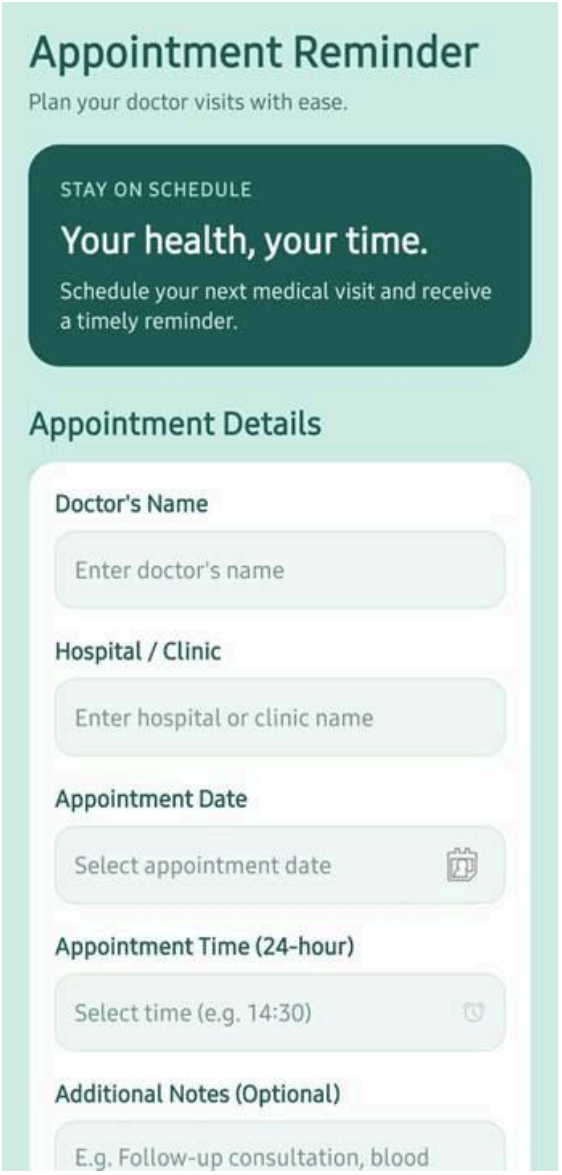


8.5 Reminder Screens and Actual Notification

Insert screenshots of the medicine reminder and appointment reminder interfaces. The images should make the date/time selection and event details visible. If the app displays a list of reminders or appointments, include a screenshot after saving the event.

An actual Android notification is especially valuable. Capture the notification generated by the test app and record the Android version and test conditions in its caption or appendix. Do not substitute a manually drawn notification. Explain that AlarmManager schedules the event and a BroadcastReceiver handles the delivered alarm, as described in the project notes.

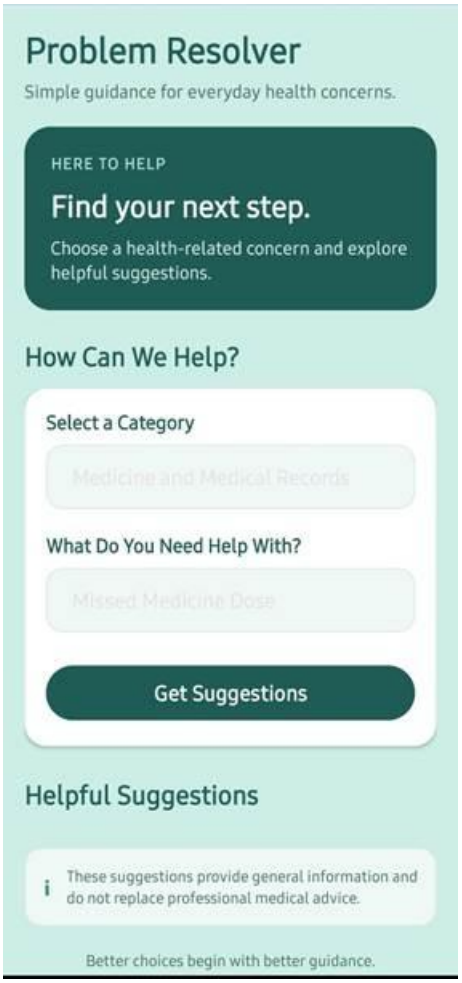




8.6 Resolve and MediBot Screens

For Problem Resolver, insert the category-selection screen and a second image showing the suggestion after a supported problem is selected. Explain that the result is predefined general guidance, not diagnosis or personalised treatment.

For MediBot, insert a supported sample question and prepared response. If suggested questions, voice input, or text-to-speech controls are visible, include appropriate screenshots. Do not label the feature as a generative AI model; the project notes describe a predefined offline response set. Avoid examples requiring personal health data.



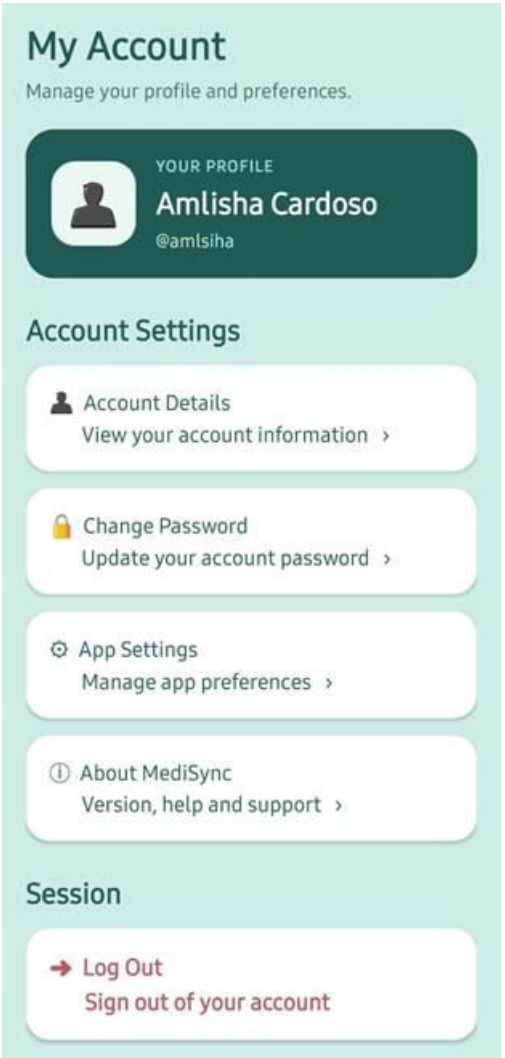


8.7 Emergency and Account Management Screens

Insert a screenshot of Emergency opening the dialer with a dummy test number. The image should show that the app opens the dialer and leaves the decision to place a call with the user; it is not necessary to place a call.

Add Account Management screenshots for account details, password change, reminder settings, or logout where available. Use a test account and do not show actual passwords. Explain the fallback when no emergency contact is configured and link the evidence to FR-12 and FR-13





8.8 Screenshot Evidence Checklist

Each screenshot should support a specific claim. A medicine list proves that the screen can display a saved entry; it does not by itself prove that the reminder fires at the selected time. A real notification is suitable evidence for that behaviour. Likewise, one account's list does not prove data separation; the evidence should show two distinct test accounts.

Check that images are legible, not distorted, and free of sensitive real data. If an image shows a failure or unexpected behaviour, explain it honestly rather than hiding it. Store originals in the project output folder if required by the assignment and use descriptive filenames.

Chapter 9: Limitations of MediSync

The current version combines useful personal information and reminder features, but has a defined scope. The supplied project notes identify limitations that should be stated clearly so that the report does not overstate the product's readiness or medical capabilities.

These limitations are opportunities for future development rather than evidence that a mini project has failed. The application demonstrates design and programming concepts while leaving cloud services, advanced security, and professional medical workflows outside the current scope.

9.1 Local-Only Storage

MediSync stores health-related information in SQLite on the device and does not currently provide automatic cloud synchronisation. Users cannot automatically access the same saved records from another device. Local data can be convenient without a network connection, but it creates a need for safe backup and recovery.

A future backup feature would need to consider user authentication, encryption in transit and at rest, access control, conflict resolution, deletion, and recovery after loss of a device. If information is uploaded to a cloud service, the user should understand what is uploaded and where it is stored. Any future cloud use of health-related data would need a privacy review appropriate to its deployment context.

9.2 MediBot and Problem Resolver Limitations

MediBot relies on prepared offline responses. It can respond to supported questions but cannot understand every way a person may phrase a concern. It is not a fully AI-powered medical chatbot. Problem Resolver similarly uses predefined categories and suggestions and does not diagnose diseases or offer personalised treatment.

Users may interpret health-related text as authoritative. Future improvements could broaden the supported question set, add multilingual interaction, and improve handling of unsupported inputs. Medical content should be reviewed for accuracy and safe wording before distribution. The current app should keep its guidance clear: the features provide general information and everyday suggestions, not emergency triage or professional advice.

9.3 Report Storage and Notification Limitations

The Medical Reports module stores metadata such as title, doctor name, hospital name, report date, and notes. It does not currently provide a complete system for uploading and viewing medical documents. Users should not expect to open a PDF or image unless a later version implements that function.

Reminder functionality depends on Android alarm and notification mechanisms. Device settings, notification permissions, background restrictions, battery optimisation, and operating-system behaviour may affect whether and when a notification appears. Reminder tests should be performed on the target Android version and should document the observed conditions.

A future version could provide document upload and secure storage, alongside better handling of edited reminders and permission denial. Those improvements would need additional testing, especially for privacy and backup behaviour.

9.4 Security and Deployment Limitations

The project notes state that MediSync includes user authentication and user-specific record separation but does not include advanced protection such as database encryption and secure cloud authentication. It should not be presented as production-ready for highly sensitive clinical records.

Before commercial deployment, the application would need a broader security review. Possible measures include secure password handling, database encryption, biometric access where appropriate, access control, backup protection, and a review of how data is retained or deleted. These measures should be based on actual security requirements and balanced with usability.

The current version also lacks a clinic-management module. It is designed for individual users rather than doctors, clinic staff, or hospital administrators. Adding those roles would require carefully controlled permissions and a clear policy for authorised access to patient information.

Chapter 10: Future Scope

MediSync provides a base that could be extended after the core functions are stable and tested. Future improvements should be chosen based on user needs, implementation effort, privacy, security, and the ability to test the new behaviour. The following directions are drawn from the future-scope section of the supplied project document.

10.1 Cloud Backup and Synchronisation

A secure cloud backup facility could let users back up and restore information and synchronise records across devices. It could help recover information after a device is lost or replaced. However, synchronisation requires a backend service, secure authentication, network error handling, conflict resolution, and consistent deletion behaviour.

The application would need to decide when local changes are uploaded, how updates are downloaded, and what happens if a record is changed on two devices. Sensitive information should be protected during transfer and storage. A cloud option should be explicit rather than silently upload health details. This is a future proposal, not part of the current locally stored version.

10.2 Medical Document Upload and Backup/Restore

The Medical Reports module could be extended to upload and open PDF or image files alongside report metadata. Documents could be associated with report title, doctor, hospital, and date, making the app more useful as a personal reference tool.

The extension would need to consider file size, allowed formats, storage location, access permissions, file corruption, and backup behaviour. A backup and restore function could allow users to export and recover their information after changing devices. Data should be protected, and the app should make clear whether deletion affects only the current device or a remote backup as well.

10.3 Improved MediBot and Multilingual Support

MediBot could be expanded to support more questions and a wider set of reviewed general information. Multilingual interaction could make it more accessible to users who prefer languages other than the default. Voice navigation and better speech feedback may also improve usability for some users.

Any move toward advanced language understanding would require safeguards against unsupported diagnosis or treatment claims. Health content should be reviewed and versioned, unsupported questions should be handled clearly, and users should be directed to professional help for serious concerns. A practical first step could be to organise prepared responses into maintainable categories, test alternate phrasings, add a clear fallback answer, and make limitations visible.

10.4 Clinic/Doctor Integration and Advanced Security

A future clinic or doctor integration could add authorised appointment management and controlled access to patient information. This would change the application from a personal organiser into a multi-role health system. It would require a backend, role-based access controls, consent rules, audit logging, and careful privacy design.

Security improvements could include stronger password handling, database encryption, biometric authentication, and role-based access control. Such measures should be selected through a threat analysis rather than added as decorative options. Data retention, recovery, and account removal should be considered together. These are future ideas and should remain labelled as future scope until implemented and tested.

10.5 Accessibility and Usability Improvements

MediSync could improve accessibility through larger text, stronger contrast, descriptive labels, simpler navigation, and voice-assisted controls. The interface should support users with different levels of mobile experience and different visual or motor capabilities.

Usability testing could ask users to save a medicine, schedule an appointment, locate an emergency contact, and find a prepared suggestion. Observers could record hesitation, unclear labels, and unnecessary steps. Accessibility should be part of testing rather than only a visual redesign.

Chapter 11: Challenges and Learning Outcomes

Developing MediSync required multiple features to work together. The main technical areas include local database management, connecting screens to CRUD operations, scheduling notifications, separating account data, and presenting general guidance with clear limitations. The exact defects encountered by the team should be added from its own development notes rather than invented.

The project demonstrates that feature completion involves validation, data persistence, navigation, error handling, and testing—not only a screen that appears on a device. It also shows why a modest, testable scope can be more valuable than a long list of features that cannot be demonstrated reliably.

11.1 Technical Learning

The project provided practice with Java-based Android development and the relationship between Java code and XML layouts. Input controls and event handlers show how a user action leads to validation, data processing, navigation, or feedback.

SQLite integration reinforced the importance of a stable data model and consistent CRUD behaviour. A record needs a clear identity, must be stored and retrieved using the right values, and should be scoped to the current user when appropriate. DatabaseHelper provides a central point for schema and query operations.

The Reminder module introduced platform behaviour beyond a simple form. Scheduling a future notification requires alarm handling and consideration of Android permissions and settings. A saved database row does not guarantee notification delivery, so the complete event path must be tested.

MediBot and Problem Resolver show how limited offline features can use prepared content. They also show why the app's capability must be described accurately: prepared responses are not the same as a generative AI model or a medically validated diagnostic service.

11.2 Software Engineering and Teamwork

MediSync demonstrates the value of modular design. Requirements are easier to implement and test when the application is divided into authentication, records, reminders, guidance, emergency, and account functions. At the same time, modules

need shared conventions for user identity, database operations, navigation, and feedback.

Documentation should track the implementation. A diagram should be updated when source code changes, a test should be marked passed only after execution, and a future feature should not be described as currently available. These practices make project communication more trustworthy.

Team collaboration includes both dividing work and integrating it. Members need to agree on field names, screen navigation, database behaviour, and error reporting. The team should add one or two actual examples of collaboration, debugging, or design decisions from its own experience to make this chapter more specific.

11.3 Team Contribution Record

The three project members are Agnel Faria (roll number 2605), Vaishnavi Sharma (roll number 2666), and Amlisha Cardoso (roll number 2608). The current project notes do not provide a verified task-by-task breakdown for each member.

Each member should list the activities actually performed. Contributions may overlap and can include requirements, UI design, Java implementation, SQLite integration, testing, diagram preparation, documentation, or integration. Evidence may include source files, commit history, dated notes, or screenshots. All three members should review the table before submission.

Member	Actual contribution(s)	Evidence / artefact
Agnel Faria (2605)	[Complete from team records]	[Source file, commit, screenshot, note]
Vaishnavi Sharma (2666)	[Complete from team records]	[Source file, commit, screenshot, note]
Amlisha Cardoso (2608)	[Complete from team records]	[Source file, commit, screenshot, note]

11.4 Improving the Development Process

The project documentation could be strengthened by maintaining dated requirement decisions, a traceability matrix, a defect log, and a repeatable test checklist from the beginning. These records make it easier to explain why a feature was chosen, what changed during development, and how the final result was verified.

A consistent naming convention for screenshots and test evidence can reduce the time needed to prepare the report. The team can also keep one approved source of truth for field names, user ID handling, and navigation destinations to prevent inconsistencies between modules.

Security and privacy should be considered early when an application handles health-related data. Even if advanced features remain outside the scope, their absence should be documented honestly and sensitive sample data should not be included in the repository or report.

Chapter 11 Summary

MediSync provided experience with native Android development, Java and XML, local relational storage, session preferences, reminder scheduling, notification handling, Intents, speech interaction, and module integration. It also reinforces that test evidence and documentation quality are part of a completed software deliverable.

Conclusion

MediSync is an Android-based health record and reminder management application developed using Java, XML, Android Studio, and SQLite. It aims to help users organise selected health-related details and remember planned activities in a single mobile interface. The concepts Record, Remind, and Resolve provide a clear structure for its user-facing features.

Record supports a basic health profile, medicine entries, medical report details, and emergency contacts. Remind provides medicine and appointment reminders using Android alarm and notification mechanisms. Resolve presents predefined suggestions, while MediBot supplies prepared offline responses with supported text and voice interaction. Emergency access opens the dialer with a saved number, and Account Management provides selected account settings.

The project demonstrates integration of UI design, Java event handling, SQLite CRUD operations, preferences, alarm scheduling, and Android Intents. User IDs connect records to accounts, and the source notes describe tests for data separation. The test register records 15 functional cases as passed.

References and Technical Resources

The project implementation summary supplied by the team is the primary source for MediSync's features, technology stack, development phases, test register, limitations, and future scope. The following official resources are relevant background documentation for the technologies named in the project.

- MediSync: Development, Implementation and Testing.
- Android Developers. Android platform documentation. https://developer.android.com/
- Android Developers. Activities and intents. https://developer.android.com/guide/components/activities/intro-activities
- Android Developers. Notifications. https://developer.android.com/develop/ui/views/notifications
- Android Developers. Alarm scheduling. https://developer.android.com/develop/background-work/services/alarms
- Android Developers. SharedPreferences and data storage. https://developer.android.com/training/data-storage/shared-preferences
- SQLite. Official documentation. https://www.sqlite.org/docs.html
- Oracle. Java documentation. https://docs.oracle.com/en/java/